

Paired Sample T test

```
import numpy as np
from scipy import stats

before = np.array([80, 85, 78, 90, 88, 84, 79])
after = np.array([82, 87, 80, 92, 89, 85, 81])

t_stat, p_value = stats.ttest_rel(before, after)

print(f"T-statistic: {t_stat:.4f}")
print(f"P-value: {p_value:.4f}")

alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: There is a significant difference before and after.")
else:
    print("Fail to reject the null hypothesis: No significant difference before and after.")
```

```
T-statistic: -9.2952
P-value: 0.0001
Reject the null hypothesis: There is a significant difference before and after.
```

Regression:

```
import pandas as pd
import statsmodels.api as sm
data = pd.read_excel("/content/only10 variable with satisfaction score.xlsx")
X =
data[["sum_physical", "sum_psychosocial", "sum_ergonomic", "sum_salary_b
enifits", "sum_workload", "sum_roldevelopment", "sum_professionaldevelom
ent", "sum_management_leadership", "sum_incivility", "sum_health_safety"]]
# All columns except the last (assuming it's the target)
y = data ["sum_total_satisfaction"]
X = sm.add_constant(X)

model = sm.OLS(y, X).fit()

print(model.summary())
from statsmodels.stats.outliers_influence import variance_inflation_factor

vif = pd.DataFrame()
vif["Features"] = X.columns
vif["VIF Factor"] = [variance_inflation_factor(X.values, i) for i in
range(X.shape[1])]

print(vif)
```

OLS Regression Results						
Dep. Variable:	sum_total_satisfaction	R-squared:	0.644			
Model:	OLS	Adj. R-squared:	0.634			
Method:	Least Squares	F-statistic:	70.40			
Date:	Fri, 21 Mar 2025	Prob (F-statistic):	4.88e-81			
Time:	07:03:17	Log-Likelihood:	-977.14			
No. Observations:	401	AIC:	1976.			
Df Residuals:	390	BIC:	2020.			
Df Model:	10					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
const	-0.7698	1.130	-0.681	0.496	-2.992	1.453
sum_physical	0.0301	0.041	0.736	0.462	-0.050	0.111
sum_psychosocial	0.2899	0.050	5.749	0.000	0.191	0.389
sum_ergonomic	0.1103	0.048	2.276	0.023	0.015	0.206
sum_salary_benifits	0.2792	0.044	6.325	0.000	0.192	0.366
sum_workload	0.0259	0.021	1.253	0.211	-0.015	0.067
sum_roldevelopment	-0.0522	0.040	-1.302	0.194	-0.131	0.027
sum_professionaldeveloment	0.5474	0.093	5.890	0.000	0.365	0.730
sum_management_leadership	0.1524	0.068	2.249	0.025	0.019	0.286
sum_incivility	0.0796	0.026	3.048	0.002	0.028	0.131
sum_health_safety	0.1431	0.039	3.653	0.000	0.066	0.220
Omnibus:	0.449	Durbin-Watson:	1.936			
Prob(Omnibus):	0.799	Jarque-Bera (JB):	0.421			
Skew:	-0.079	Prob(JB):	0.810			
Kurtosis:	2.989	Cond. No.	480.			

Machine Learning

```

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score, accuracy_score
from sklearn.preprocessing import StandardScaler

import matplotlib.pyplot as plt

data = pd.read_excel("/content/only10 variable with satisfaction score.xlsx")

X =
data[["sum_physical", "sum_psychosocial", "sum_ergonomic", "sum_salary_benifits", "sum_workload", "sum_
m_roldevelopment", "sum_professionaldeveloment", "sum_management_leadership", "sum_incivility", "
sum_health_safety"]] # All columns except the last (assuming it's the target)
y = data ["sum_total_satisfaction"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = LinearRegression()
model.fit(X_train_scaled, y_train)

y_pred = model.predict(X_test_scaled)

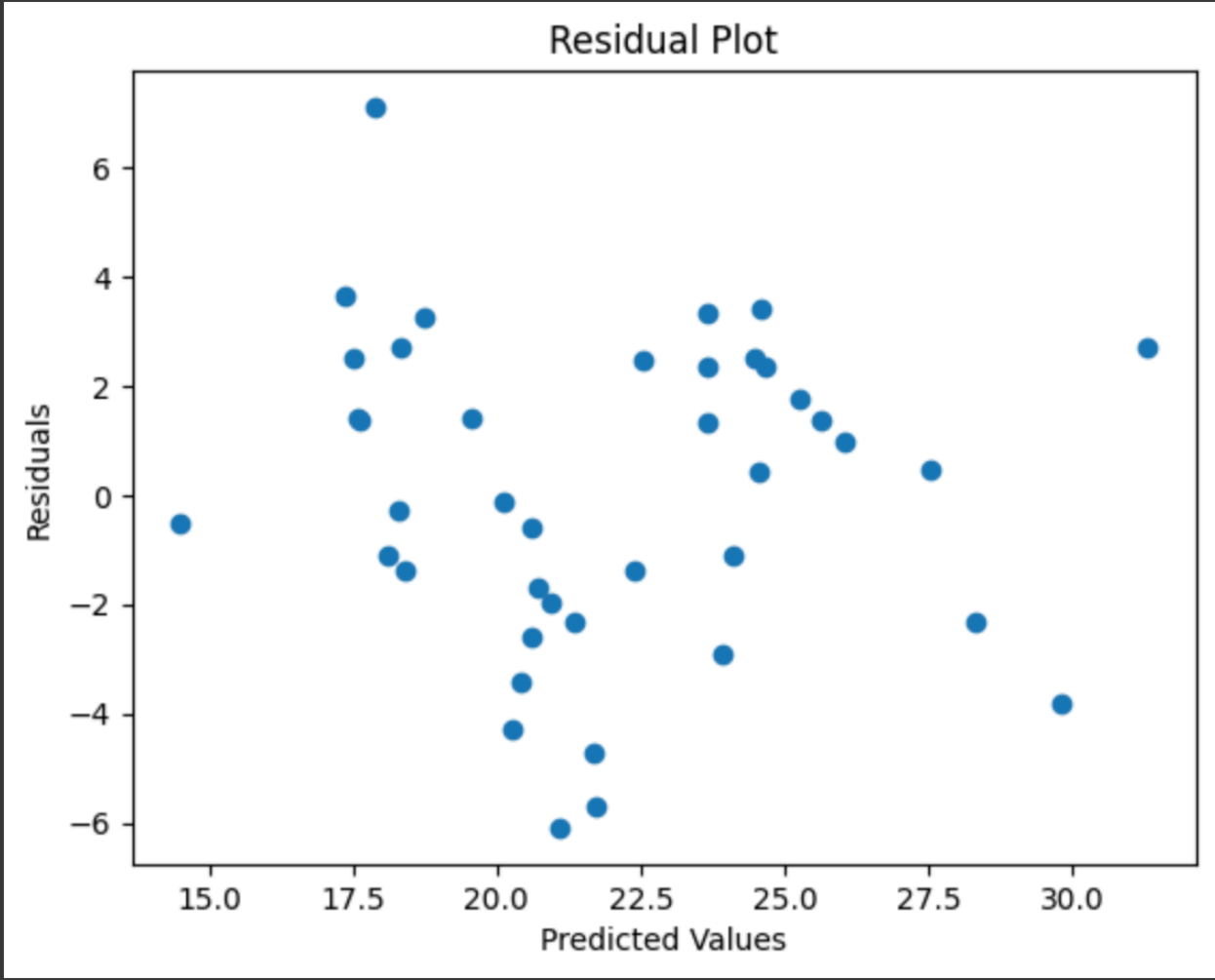
mse = mean_squared_error(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print(f"Mean Squared Error: {mse}")
print(f"Mean Absolute Error: {mae}")
print(f"R^2 Score: {r2}")
threshold = 0.10 # 10% threshold
correct_predictions = np.abs((y_test - y_pred) / y_test) < threshold
accuracy = np.mean(correct_predictions)
print(f"Accuracy Score: {accuracy}")

cv_mse = cross_val_score(model, X, y, cv=10, scoring='neg_mean_squared_error')
cv_mae = cross_val_score(model, X, y, cv=10, scoring='neg_mean_absolute_error')
cv_r2 = cross_val_score(model, X, y, cv=10, scoring='r2')
# cv_accuracy=cross_val_score(model,X,y,cv=6,scoring='accuracy_score')
print(f"Cross-Validation MSE: {-np.mean(cv_mse)}")
print(f"Cross-Validation MAE: {-np.mean(cv_mae)}")
print(f"Cross-Validation R^2: {np.mean(cv_r2)}")

residuals = y_test - y_pred
plt.scatter(y_pred, residuals)
plt.xlabel('Predicted Values')
plt.ylabel('Residuals')
plt.title('Residual Plot')
plt.show()

```

Mean Squared Error: 8.083549091225024
Mean Absolute Error: 2.369553541770003
R^2 Score: 0.6006452118277417
Accuracy Score: 0.5853658536585366
Cross-Validation MSE: 9.913058317483443
Cross-Validation MAE: 2.318399367709824
Cross-Validation R^2: 0.5033747560079033



Regression

```
import statsmodels.api as sm
import pandas as pd
# Example data
data = {
    'X1': [10, 22, 33, 54, 55],
    'X2': [2, 3, 4, 5, 6],
    'Y': [11, 13, 12, 15, 14]
}
df = pd.DataFrame(data)

# Dependent variable
Y = df['Y']

# Independent variables (add a constant term for the intercept)
X = df[['X1', 'X2']]
X = sm.add_constant(X)
# Fit the model
model = sm.OLS(Y, X).fit()
# Print the model summary
print(model.summary())
```

OLS Regression Results						
Dep. Variable:	Y	R-squared:	0.858			
Model:	OLS	Adj. R-squared:	0.717			
Method:	Least Squares	F-statistic:	6.063			
Date:	Fri, 18 Apr 2025	Prob (F-statistic):	0.142			
Time:	11:29:35	Log-Likelihood:	-3.9405			
No. Observations:	5	AIC:	13.88			
Df Residuals:	2	BIC:	12.71			
Df Model:	2					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
const	12.2659	1.801	6.809	0.021	4.515	20.017
X1	0.1761	0.100	1.756	0.221	-0.255	0.608
X2	-1.3489	1.252	-1.077	0.394	-6.736	4.038
Omnibus:	nan	Durbin-Watson:	3.180			
Prob(Omnibus):	nan	Jarque-Bera (JB):	0.342			
Skew:	0.519	Prob(JB):	0.843			
Kurtosis:	2.251	Cond. No.	223.			

Independent Sample T test

```
import numpy as np
from scipy import stats

# Sample data for two independent groups
group1 = np.array([23, 21, 18, 25, 27, 22, 20])
group2 = np.array([30, 28, 35, 33, 27, 29, 31])

# Perform independent t-test
t_stat, p_value = stats.ttest_ind(group1, group2)
### t_stat, p_value = stats.ttest_ind(group1, group2, equal_var=False)##

# Output results
print(f"T-statistic: {t_stat:.4f}")
print(f"P-value: {p_value:.4f}")

# Interpretation
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: The means are significantly different.")
else:
    print("Fail to reject the null hypothesis: No significant difference between means.")
```

```
T-statistic: -5.1962
P-value: 0.0002
Reject the null hypothesis: The means are significantly different.
```